LLARS - Script Creator & Framework Guide

Current Creator API Edition

Current for LLARS v2

Scope - This guide documents the public LLARS project as it operates in v2. Follow the shared framework and the current GUI/API behavior described here.

Contents

1. Before You Start
2. Framework Ownership
3. Project Structure
4. Templates and Script Generator
5. Script Skeleton and Bootstrap
6. Config.ini and Typed Setup
7. Creator Configuration API
8. RunCount and Run Context
9. Runtime Estimation
10. Focus and Input Safety
11. Clicks and Keyboard Input
12. Status, Sleeps, and Random Sleep
13. Managed Timers and Timed Scripts
14. Pixel and Color APIs
15. Optional Features and Dependencies
16. Developer Mode
17. Diagnostics API
19. API Reference
20. Common Creator Mistakes
21. The Simple Version

1. Before You Start

This guide is for creators writing, converting, maintaining, or contributing scripts that live in the current LLARS project. Use AutoHotkey v1.1.37.02 syntax and start from an existing current template whenever possible.

Project rule - Game-facing physical input in public Scripts and Templates must use LLARS shared input APIs. Do not add raw Send, Click, MouseClick, MouseMove, direct AHI calls, or direct Win32 input injection to those folders.

2. Framework Ownership

Keep task-specific decisions in the script. Let LLARS own shared lifecycle and safety.

Script owns	Framework owns
Activity sequence, target choices, activity-specific detection	GUI lifecycle, start/pause/exit state
Activity-specific Config.ini metadata	Typed config validation and setup UI
When an action should occur	RuneScape window binding/focus recovery
Activity-specific status/detection messages	Input routing, AHI fallback, diagnostics stream
Callback logic	Managed timers, cleanup, support reporting

3. Project Structure

```
LLARS v2/
  Script Selector.ahk
  LLARS Config.ini
  VERSION
  README.md
  Core/
    LLARS.ahk
    LLARS_Core.ahk
    LLARS_GUI.ahk
    LLARS_Labels.ahk
    LLARS_Runtime.ahk
    LLARS_CreatorAPI.ahk
    LLARS_AHI.ahk
    LLARS_Update.ahk
    LLARS_ScriptBootstrap.ahk
  Scripts/
  Templates/
  Assets/
  Backup/
  Documentation/
  Vendor/
```

Core is shared framework code. Scripts contains user-facing activities. Templates contains starter scripts and the Script Generator. Vendor contains bundled third-party runtime components and licenses. Backup and Documentation are public user/creator resources.

4. Templates and Script Generator

- Timer - one timed session with optional managed repeating callback.
- MultiTimer - multiple independently scheduled callbacks.
- Color - one configured pixel/color state with reset gating.
- MultiColor - multiple configured colors mapped to action coordinates.
- Script Generator - creates a starter from the supported template model.

5. Script Skeleton and Bootstrap

```
#Requires AutoHotkey v1.1.37.02
#SingleInstance Force
#Persistent

LLARS_SCRIPT_TYPE := "Timer"

if !LLARS_FrameworkAvailable()
    LLARS_FrameworkError()

LLARS_Initialize()
return
```

Current scripts use the shared bootstrap/framework include path. Keep the complete project structure intact instead of copying framework files into each script.

6. Config.ini and Typed Setup

Typed metadata drives the setup GUI, validation, reset behavior, dependency handling, and diagnostics. Common types include coordinate, hotkey, and color. Timer/sleep sections use option/min/max conventions; scripts may also read typed strings, integers, numbers, booleans, or choices.

Required values - If a setting is required, blank or invalid values should fail clearly. Optional dependencies should be disabled with their option metadata rather than bypassing validation in script code.

7. Creator Configuration API

Function	Use
LLARS_ConfigRead(...)	Generic key read with explicit default/scope.
LLARS_ConfigReadString / Integer / Number / Bool / Choice	Typed reads with validation.
LLARS_ConfigReadHotkey / Color	Read validated typed hotkey/color values.
LLARS_ConfigEnabled(section, default, scope)	Evaluate an optional section and its dependency state.
LLARS_ConfigReadRange(...)	Read min/max range.
LLARS_ConfigReadPoint(...)	Read one configured coordinate point.

scope defaults to "script". Use the root scope only for settings intentionally owned by the shared LLARS Config.ini.

8. RunCount and Run Context

Use LLARS_RunCount(Func("YourCallback")) for current RunCount scripts. The callback receives a context object so first-run and iteration state do not need ad-hoc globals.

```
RunTask(ctx)
{
    LLARS_SetStatus("Working", "Run " . ctx.Iteration . "/" . ctx.Total)
    if (ctx.IsFirst)
        LLARS_LogEvent("Run started", "First iteration")

    LLARS_Click("Action Location")
    LLARS_Sleep("Action Delay", ctx.IsLast)
}
```

9. Runtime Estimation

RunCount estimation analyzes the script editing region and current LLARS timing API. Declare normal waits with LLARS_Sleep/LLARS_RandomSleep so the estimate represents actual behavior. Existing compatible direct IniRead/Sleep patterns are recognized, but current scripts should prefer the typed timing API.

10. Focus and Input Safety

At run start LLARS binds the selected RuneScape client. Game-facing Creator API actions validate that window and wait/recover focus where appropriate. LLARS_WaitForRuneScape exists for specialized non-input work that must only happen while RuneScape is the active run target; it is not a reason to send raw physical input.

11. Clicks and Keyboard Input

Function	Use
LLARS_Click(section, button, scope)	Configured coordinate click using shared input routing.
LLARS_RedClick / LLARS_RedClickRetry	Configured click with RuneScape red-click confirmation behavior.
LLARS_ClickPoint / LLARS_RedClickPoint	Click an already-resolved target point through shared routing.
LLARS_PressKey(keyName)	Press a named key through the active backend.
LLARS_PressHotkey(section, key, scope)	Press a configured hotkey through the active backend.

Do not write backend-specific logic in activity scripts. AHI/Standard routing and per-device fallback belong to Core.

12. Status, Sleeps, and Random Sleep

Function	Use
LLARS_SetStatus(status, activity)	Update main GUI and diagnostic status.
LLARS_Sleep(section, final, scope)	Use configured min/max timing with run-state handling.
LLARS_RandomSleep()	Run the shared optional long-break behavior.

13. Managed Timers and Timed Scripts

Prefer callback timers because LLARS tracks and cleans them up automatically.

```
ScriptTimerID := LLARS_TimerRepeat("Script Timer", Func("ScriptTimer"))

ScriptTimer()
{
    if !LLARS_RunActive()
        return
    LLARS_SetStatus("Working", "Timer Action")
    LLARS_Click("Action Location")
}

; Stop one or all managed timers when needed:
LLARS_TimerStop(ScriptTimerID)
LLARS_TimerStopAll()
```

LLARS_TimerOnce schedules one callback. LLARS_TimerRepeat chooses a fresh configured interval after each callback. The label compatibility API remains available for maintained code, but Func callbacks are the current default.

14. Pixel and Color APIs

Function	Use
LLARS_PixelColor	Read a configured RuneScape pixel safely.
LLARS_PixelMatches / LLARS_PixelMatchesAny	Match one or several configured colors.
LLARS_PixelMatchesWithinTolerance	Match with RGB tolerance.
LLARS_FindColorCenterOut	Find an enabled configured color from client center outward.
LLARS_VerifyColorTarget	Re-verify a resolved color target before acting.

LLARS_Color... helpers	Shared capture, clustering, density, signature, and tracking utilities for advanced color workflows.

Use public LLARS capture/color helpers instead of adding direct Win32 capture/input calls to a script. Detection helpers automatically contribute framework-visible diagnostic state where appropriate.

15. Optional Features and Dependencies

Use LLARS_ConfigEnabled() for optional sections and model dependencies in Config.ini metadata. Disabled dependencies should appear as Not Required in Configuration Status instead of producing false missing-config errors.

16. Developer Mode

Developer Mode is an observation surface, not a runtime dependency. Full Mode includes active hotkeys, Live Inspector, Configuration Diagnostics, Detection, and Recent Framework Actions; Lightweight Mode focuses on recent actions. Creator scripts must behave correctly when the diagnostic UI is closed.

17. Diagnostics API

Function	Purpose
LLARS_LogEvent(summary, detail := "")	Record a meaningful creator event in Recent Framework Actions/support diagnostics.
LLARS_Warn(title, whatHappened, whatLLARSDid := "", whatUserShouldDo := "", detail := "", showMessage := true)	Record/show a structured warning.
LLARS_Error(...)	Record/show a structured error with the same fields.
LLARS_Recovered(summary, detail := "")	Record a successful recovery/fallback.
LLARS_SetDetection(name, state, confidence := "", threshold := "", detail := "", logChange := true)	Publish current detection state for Developer Mode/support.

```
LLARS_SetDetection("Deposit", "Searching", "", "", "Center-out scan")
if (found)
    LLARS_SetDetection("Deposit", "Matched", score, minimumScore, "Target acquired")
else
    LLARS_Warn("Deposit not found", "No valid target matched.", "No click was sent.", "Check
layout and configured colors.", "", false)
```

Current logger - The current public diagnostics API uses LLARS_LogEvent(). A generic logger named Log is not part of the Creator API.

18. Contribution Boundaries

- Run the script with Developer Mode closed as well as open.
- Verify Configuration Status has no unexpected Required/Missing items.
- Verify pause/resume/exit and timer cleanup.
- Verify the chosen RuneScape client stays the only game target.
- Exercise AHI and Standard routes where the action uses physical input.
- Do not add raw physical input commands, direct AHI calls, or direct Win32 input injection under Scripts or Templates.

- Keep public source/comments focused on product behavior and maintainable rationale.

19. API Reference

Area	Primary current helpers
Config	LLARS_ConfigRead, LLARS_ConfigReadString, LLARS_ConfigReadInteger, LLARS_ConfigReadNumber, LLARS_ConfigReadBool, LLARS_ConfigReadChoice, LLARS_ConfigReadHotkey, LLARS_ConfigReadColor, LLARS_ConfigEnabled, LLARS_ConfigReadRange, LLARS_ConfigReadPoint
Run/lifecycle	LLARS_RunActive, LLARS_RunCount, LLARS_WaitForRuneScape
Timing	LLARS_Sleep, LLARS_TimerInterval, LLARS_TimerOnce, LLARS_TimerRepeat, LLARS_TimerStop, LLARS_TimerStopAll
Input	LLARS_Click, LLARS_RedClick, LLARS_RedClickRetry, LLARS_ClickPoint, LLARS_RedClickPoint, LLARS_RedClickRetryPoint, LLARS_PressKey, LLARS_PressHotkey
Status/diagnostics	LLARS_SetStatus, LLARS_LogEvent, LLARS_Warn, LLARS_Error, LLARS_Recovered, LLARS_SetDetection
Pixel/color	LLARS_PixelColor, LLARS_PixelMatches, LLARS_PixelMatchesAny, LLARS_PixelChangedFrom, LLARS_PixelMatchesWithinTolerance, LLARS_FindColorCenterOut, LLARS_VerifyColorTarget, LLARS_Color* advanced helpers

Use the Script Creator Library for copy/paste recipes and inspect Core/LLARS_CreatorAPI.ahk for exact signatures of advanced helpers.

20. Common Creator Mistakes

- Using raw Send/Click/MouseMove because it seems shorter. This bypasses routing, focus safety, fallback, and diagnostics.
- Inventing a helper name from an older guide. Use the current API names in Core/LLARS_CreatorAPI.ahk.
- Making Developer Mode required for behavior. It must remain optional.
- Reading typed config with ad-hoc parsing when a typed LLARS_ConfigRead* helper already exists.
- Starting unmanaged repeating work without stopping it on exit/end. Prefer managed timer APIs.
- Adding new top-level files when an existing project location already fits.

21. The Simple Version

1. Start from a current Template.
2. Declare typed Config.ini settings.
3. Use LLARS_RunCount or the timed lifecycle that matches the script.
4. Use LLARS_Click / LLARS_Press* / LLARS_Sleep / managed timers and shared color APIs.
5. Publish meaningful status and detection state with LLARS_SetStatus and the diagnostics API.
6. Keep physical input inside the shared LLARS input layer.
7. Verify lifecycle, config, focus, and both input routes.

Creator Configuration API

Shared writes and section enumeration

Project rule - Scripts and Templates use the shared LLARS configuration API.
Native IniRead / IniWrite / IniDelete remain implementation details of approved Core/config code.

22. Config Writes and Section Enumeration

The Creator API includes two shared primitives for cases that need to persist settings or inspect dynamic section names. Both use the same script/root path resolution as the existing read helpers.

LLARS_ConfigWrite(section, key, value, scope := "script")

- Writes one key through the shared config path resolver and returns true on success.
- Use scope="root" only for settings intentionally owned by LLARS Config.ini.
- Write failures are surfaced through the normal creator configuration error path instead of silently succeeding.

LLARS_ConfigSections(scope := "script", default := "ERROR")

- Returns the raw newline-delimited section list from the selected config file.
- Returns the supplied default when the config file or section enumeration is unavailable.
- Use this for templates or scripts whose configuration intentionally contains a variable set of named sections.

Example

```
if !LLARS_ConfigWrite("Timer", "min", newMin)
    return

sections := LLARS_ConfigSections()
if (sections = "ERROR")
    return
Loop, Parse, sections, `n, `r
    sectionName := A_LoopField
```

Prefer the existing typed LLARS_ConfigRead* helpers when reading known values. These two primitives close the remaining write/enumeration gap without creating a second configuration system.